Buổi 8: Lập trình kết nối mạng

Nội dung chính:

8.1. Kết nối mạng cơ bản với HttpURLConnection.

8.2. Tương tác RESTful API sử dụng Retrofit.

8.3. Quản lý yêu cầu không đồng bộ với Coroutine.

8.4. UDF và Firebase trong Android

Mục tiêu cần đạt được:

Hiểu cách kết nối mạng trong ứng dụng Android, từ cơ bản đến nâng cao.

Biết cách sử dụng Retrofit để làm việc với RESTful API.

Vận dụng Coroutine để xử lý các yêu cầu mạng không đồng bộ, tối ưu hóa hiệu năng ứng dụng.

8.1. Kết nối mạng cơ bản với HttpURLConnection.

8.1.1. Tổng quát về kết nối mạng

Kết nối và sử dụng tài nguyên mạng

Android hỗ trợ các hàm API cho phép kết nối với các mạng truyền thông phổ biến như Wifi, bluetooth, NFC,... Phần này sẽ trình bày các hàm API hỗ trợ lập trình kết nối ứng dụng Android với mạng Internet.

Các ứng dụng Android có nhu cầu truy xuất chủ yếu đến các dịch vụ web (web services) thông qua giao thức HTTP. Các dịch vụ web chia làm hai dạng là XML/SOAP và RESTful. Các dịch vụ web dạng XML/SOAP có tính năng đầy đủ hơn nhưng yêu cầu dung lượng lớn hơn, còn các dịch vụ dạng RESTful thì khá nhẹ nhưng các tính năng được cắt giảm để đơn giản hơn và không sử dụng định dạng dữ liệu XML. RESTful sử dụng phương pháp JSON (JavaScript Object Notation) và các định dạng dữ liệu khác cho WEB.

Phần sau đây sẽ trình bày cách truy xuất dịch vụ web dạng RESTful. Android cung cấp thư viện Apache HttpClient hỗ trợ lập trình với giao thức HTTP ở mức cao và hỗ trợ sử dụng phương hai phương thức GET and POST.

Trước hết, để ứng dụng Android có thể truy cập vào mạng người lập trình cần cấp quyền truy cập trong file manifest như sau:

```
<uses-permission android:name="android.permission.INTERNET"/>
```

Quyền kiểm tra trạng thái kết nối mạng của thiết bị được kiểm soát bởi thẻ:

```
<uses-permission android:name="android.permission.
ACCESS_NETWORK_STATE"/>
```

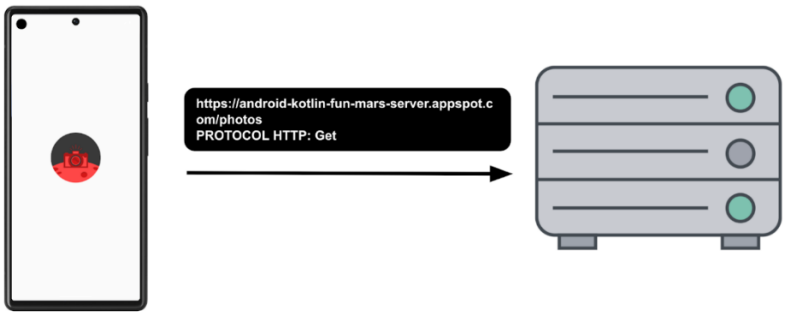
Yêu cầu dịch vụ web

Mỗi yêu cầu dịch vụ web chứa một URI và được gửi tới máy chủ thông qua giao thức HTTP mà các trình duyệt web (như Chrome) sử dụng. Các yêu cầu HTTP bao gồm một thao tác để thông báo cho máy chủ biết cần thực hiện điều gì.

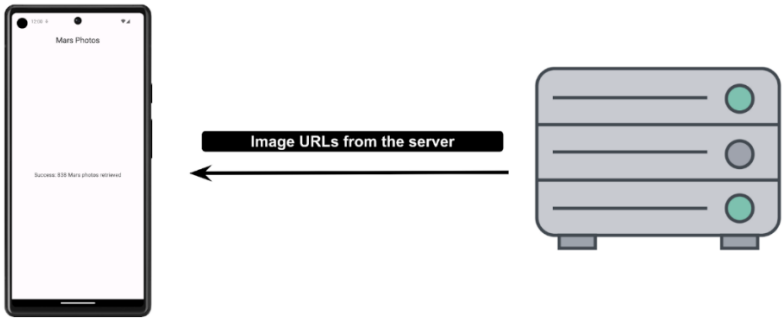
Một số thao tác HTTP phổ biến bao gồm:

- GET: truy xuất dữ liệu từ máy chủ.
- POST: tạo dữ liệu mới trên máy chủ.
- PUT: cập nhật dữ liệu hiện có trên máy chủ.
- DELETE: xoá dữ liệu khỏi máy chủ.

Ví dụ, trong ảnh dưới đây ứng dụng sẽ gửi yêu cầu HTTP GET tới dịch vụ máy chủ (https://android-kotlin-fun-mars-server.appspot.com/photos) để yêu cầu cung cấp thông tin về ảnh sao Hỏa, sau đó máy chủ sẽ trả về phản hồi chứa URL của hình ảnh.

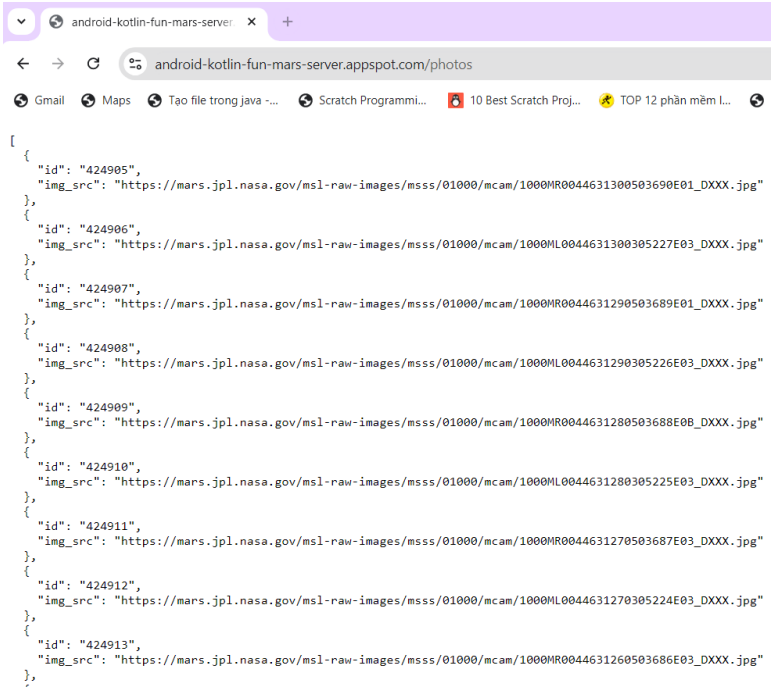


Hình 8.1: HTTP GET Request



Hình 8.2: HTTP GET Response

Phản hồi của một dịch vụ web được định dạng theo một trong các định dạng dữ liệu phổ biến, chẳng hạn như XML (Ngôn ngữ đánh dấu mở rộng) hoặc JSON (Ký hiệu đối tượng JavaScript). Định dạng JSON biểu thị dữ liệu có cấu trúc trong các cặp khoá–giá trị.



Hình 8.3: Ví dụ về Json

Sử dụng thư viện thuộc gói java.net

Gói java.net bao gồm các lớp cho phép lập trình viên thực hiện các tác vụ liên quan đến mạng, như:

- Tạo và quản lý kết nối mạng: Gửi và nhận dữ liệu qua các giao thức mạng như HTTP, TCP, UDP.
- Xử lý địa chỉ IP: Chuyển đổi giữa tên miền và địa chỉ IP, xác định thông tin về mạng.
- Kết nối đến máy chủ: Gửi yêu cầu và nhận phản hồi từ máy chủ.

Các Lớp Chính Trong Gói java.net

- URL: Lớp này đại diện cho một URL (Uniform Resource Locator) và cho phép bạn đọc dữ liệu từ một địa chỉ cụ thể.
- HttpURLConnection: Lớp này cho phép thực hiện các yêu cầu HTTP và nhận phản hồi từ máy chủ. Ta có thể thiết lập phương thức (GET, POST, PUT, DELETE) và gửi dữ liệu đi.
- …

Cách Sử Dụng Trong Ứng Dụng Android

- Kết Nối Mạng: Sử dụng HttpURLConnection để gửi yêu cầu HTTP đến máy chủ và nhận phản hồi. Điều này hữu ích cho việc tải dữ liệu từ API.
- Xử Lý Dữ Liệu: Sau khi nhận phản hồi, phân tích dữ liệu JSON hoặc XML để hiển thị trong ứng dụng.

- Quản Lý Kết Nối: Sử dụng Socket và ServerSocket để xây dựng các ứng dụng.

Ví Dụ Kết Nối Mạng

Dưới đây là một ví dụ đơn giản về việc sử dụng HttpURLConnection để thực hiện yêu cầu GET:

Làm việc với HttpClient

Lớp HttpClient cung cấp phương thức GET hỗ trợ kết nối đến dịch vụ web. Việc truy xuất đến dịch vụ Web sử dụng lớp HttpClient theo cú pháp:

```
import java.net.HttpURLConnection
import java.net.URL
val url = URL(link)
val connection = url.openConnection() as HttpURLConnection
connection.requestMethod = "GET"
```

Tham số link là đường dẫn đến địa chỉ của Server, các thí dụ phần tiếp theo sẽ giải thích rõ hơn về cách sử dụng lớp HttpClient.

Để sử dụng các lớp hỗ trợ lập trình PHP trong ứng dụng Android người lập trình phải thêm đoạn lệnh sau vào file build.gradle:

```
android {
useLibrary 'org.apache.http.legacy'
}
```

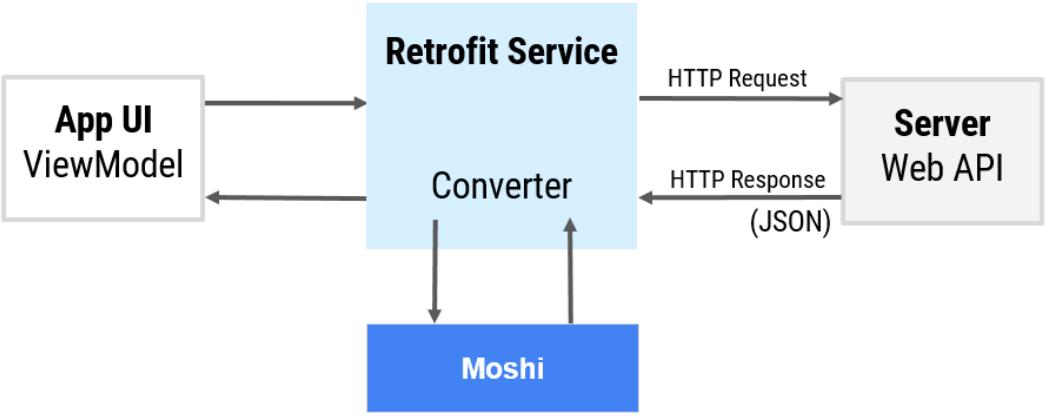
Làm việc với UrlConnection

Lớp UrlConnection kết hợp với lớp URLEncoder cung cấp các phương thức kết nối đến dịch vụ web qua phương thức POST và cho phép mã hoá các thông tin như sau:

```
import java.net.URL
import java.net.URLEncoder
val url = URL(link)
val data = "${URLEncoder.encode("username", "UTF-8")}=" +
           "${URLEncoder.encode(username, "UTF-8")}" +
           "&${URLEncoder.encode("password", "UTF-8")}=" +
           "${URLEncoder.encode(password, "UTF-8")}"
val connection = url.openConnection()
```

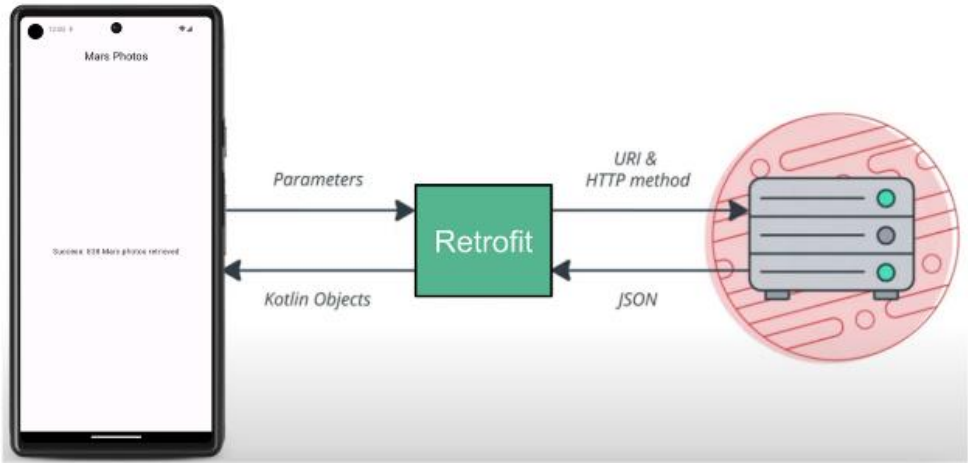
8.2. Tương tác RESTful API sử dụng Retrofit

Thư viện bên ngoài hoặc thư viện bên thứ ba giống như phần mở rộng cho các API Android cốt lõi.



Hình 8.4: Kết nối thư viện ngoài

Retrofit là một thư viện phổ biến trong Android dùng để kết nối ứng dụng với các dịch vụ web RESTful. Nó giúp việc gửi các yêu cầu HTTP và nhận phản hồi từ máy chủ trở nên đơn giản và dễ quản lý. Dưới đây là hướng dẫn chi tiết về cách sử dụng Retrofit.



Hình 8.5: Retrofit

Thêm phần phụ thuộc Retrofit

Android Gradle cho phép thêm các thư viện bên ngoài vào dự án. Ngoài phần phụ thuộc thư viện, ta cũng cần đưa vị trí lưu trữ thư viện vào kho lưu trữ.

Mở tệp gradle cấp module build.gradle.kts (Module :app).

Trong phần dependencies, hãy thêm những dòng sau cho các thư viện Retrofit:

```
// Retrofit
implementation("com.squareup.retrofit2:retrofit:2.9.0")
// Retrofit with Scalar Converter
implementation("com.squareup.retrofit2:converter-scalars:2.9.0")
```

Cả hai thư viện hoạt động cùng nhau. Phần phụ thuộc đầu tiên là dành cho chính thư viện Retrofit2 và phần phụ thuộc thứ hai là dành cho bộ chuyển đổi vô hướng Retrofit (Retrofit scalar converter). Retrofit2 là phiên bản cập nhật của thư viện Retrofit.

Bộ chuyển đổi vô hướng này cho phép Retrofit trả về kết quả JSON dưới dạng String. JSON là định dạng lưu trữ và truyền dữ liệu giữa ứng dụng và máy chủ.

Nhấp vào Đồng bộ hoá ngay (Sync Now) để xây dựng lại dự án với các phần phụ thuộc mới.

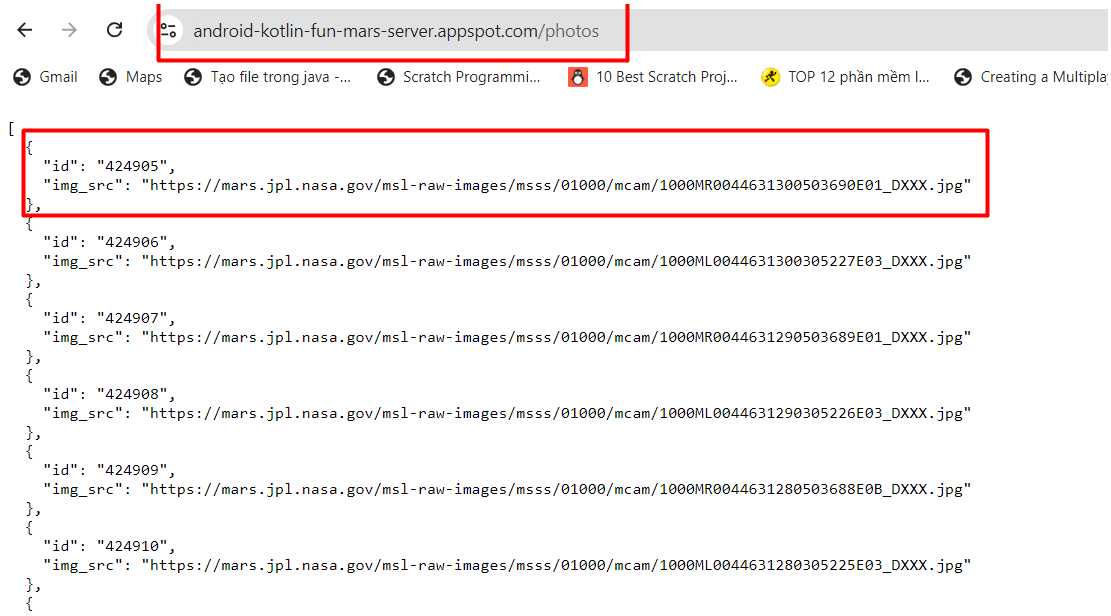
Hình 8.6: Cấu hình Retrofit

```
// Retrofit
implementation("com.squareup.retrofit2:retrofit:2.9.0")
// Retrofit with Scalar Converter
implementation("com.squareup.retrofit2:converter-scalars:2.9.0")
//làm việc với json bằng thư viện gson
implementation("com.squareup.retrofit2:converter-gson:2.9.0")
//thêm thư viện để có thể dùng đối tượng ảnh với tài nguyên là url
implementation("io.coil-kt:coil-compose:2.4.0")
//sử dụng ViewModel
    implementation ("androidx.lifecycle:lifecycle-viewmodel-
compose:2.6.1")
```

Bước 2: Tạo một lớp để lưu dữ liệu lấy về từ dịch vụ web (lớp kotlin)

```
data class Photo(
    val id: String,
    @SerializedName("img_src") val image: String
)
```

annotation @SerializedName mô tả trường này tương đương với dữ liệu có tên img_src trên server:



https://android-kotlin-fun-mars-server.appspot.com/photos

Bước 3: Tạo một interface, interface này sẽ cung cấp các hàm để làm việc với các dịch vụ của web, người lập trình chỉ cần mô tả phương thức, kiểu trả về còn những việc khác Retrofit sẽ xử lý: (interface trong thư mục kava+kotlin)

phương thức get với đường dẫn dịch vụ là photos : //https://android-kotlin-fun-mars-server.appspot.com/photos

```
import retrofit2.http.GET
interface MyPhoToService {
    @GET("photos") //
    suspend fun getPhoto() : List<Photo>
}
```

Bước 4: Tạo 1 đối tượng khởi tạo 1 lần để tạo kết nối đến server (chỉ tạo 1 đối tượng duy nhất): bạn có thể tạo 1 file riêng rồi tạo đối tượng trong file này

```
import retrofit2.Retrofit
import retrofit2.converter.gson.GsonConverterFactory
object myBuilder{
    fun create(): Retrofit{
        val retrofit = Retrofit.Builder()
            .baseUrl("https://android-kotlin-fun-mars-
server.appspot.com/")
            .addConverterFactory(GsonConverterFactory.create())
            .build()
        return retrofit
    }
}
```

Bước 5. kết nối :

```
class MainActivity : AppCompatActivity() {
    lateinit var imageView: ImageView
```

```
        override fun onCreate(savedInstanceState: Bundle?) {
            super.onCreate(savedInstanceState)
            enableEdgeToEdge()
            setContentView(R.layout.activity_main)
            imageView = findViewById(R.id.iv)
            GlobalScope.launch
    {
                val connect =
                 myBuilder.create().create(MyPhoToService::class.java)
      }
    }
}
```

8.3. Quản lý yêu cầu không đồng bộ với Coroutine

Coroutines trong Kotlin là một công cụ mạnh mẽ giúp đơn giản hóa lập trình không đồng bộ và xử lý các tác vụ dài. Sử dụng coroutines giúp tránh việc sử dụng callback phức tạp và đảm bảo mã dễ đọc, dễ bảo trì hơn.

Khái niệm cơ bản về Coroutine

Coroutine cho phép thực hiện các tác vụ nặng (chẳng hạn như yêu cầu mạng hoặc đọc/ghi vào cơ sở dữ liệu) một cách không đồng bộ, giúp giao diện người dùng không bị "đóng băng". Coroutines hoạt động với luồng nền nhưng lại cung cấp các phương pháp dễ hiểu và gọn gàng để làm việc không đồng bộ.

Cách thêm Coroutine vào dự án

Để sử dụng coroutine, cần thêm các thư viện cần thiết vào tệp build.gradle của module:

```
dependencies {
    implementation "org.jetbrains.kotlinx:kotlinx-coroutines-
    core:1.5.0"
    implementation "org.jetbrains.kotlinx:kotlinx-coroutines-
    android:1.5.0"
}
```

Sử dụng Coroutine trong Android

Để khởi chạy một coroutine trong Android, sử dụng CoroutineScope. Có thể tạo một coroutine trên luồng chính bằng cách sử dụng MainScope() hoặc trong một ViewModel với viewModelScope.

```
import kotlinx.coroutines.*
fun fetchMarsPhotos() {
    GlobalScope.launch(Dispatchers.Main) {
        val photos = withContext(Dispatchers.IO) {
            // Giả sử đây là một hàm gọi mạng lấy ảnh sao Hoả
            fetchPhotosFromNetwork()
        }
        // Cập nhật giao diện người dùng với dữ liệu ảnh
```

```
            updateUI(photos)
        }
    }
```

- GlobalScope.launch: Khởi chạy một coroutine ở phạm vi toàn cục. Thông thường, tránh sử dụng phạm vi toàn cục mà nên sử dụng các phạm vi gắn liền với vòng đời của thành phần (như viewModelScope hoặc lifecycleScope).
- Dispatchers.Main: Chỉ định coroutine sẽ chạy trên luồng chính, phù hợp cho các tác vụ liên quan đến UI.
- withContext(Dispatchers.IO): Chuyển sang luồng I/O để thực hiện các tác vụ nặng như gọi mạng hoặc đọc/ghi cơ sở dữ liệu.

Khởi chạy coroutine trong ViewModel

Sử dụng viewModelScope để khởi chạy coroutine trong ViewModel, giúp tự động hủy coroutine khi ViewModel bị hủy, tránh rò rỉ bộ nhớ.

```
import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import kotlinx.coroutines.launch
class MarsViewModel : ViewModel() {
    fun fetchMarsPhotos() {
        viewModelScope.launch {
            val photos = withContext(Dispatchers.IO) {
                fetchPhotosFromNetwork()
            }
            updateUI(photos)
        }
    }
}
```

Các loại Coroutine Scope

- **GlobalScope**: Phạm vi toàn cục, tồn tại trong suốt thời gian sống của ứng dụng. Nên tránh sử dụng vì có thể gây rò rỉ bộ nhớ.
- **viewModelScope**: Dành cho ViewModel, các coroutine khởi chạy trong phạm vi này sẽ tự động bị hủy khi ViewModel bị hủy.
- **lifecycleScope**: Dành cho các Activity hoặc Fragment, các coroutine sẽ tự động hủy khi vòng đời của Activity hoặc Fragment kết thúc.

Các loại Dispatcher

- **Dispatchers.Main**: Dành cho các tác vụ cần tương tác với UI.
- **Dispatchers.IO**: Dành cho các tác vụ I/O như đọc/ghi file hoặc gọi API mạng.
- **Dispatchers.Default**: Dành cho các tác vụ tính toán nặng, không liên quan đến I/O.

Sử dụng async để chạy song song

Đôi khi cần thực hiện nhiều tác vụ song song và lấy kết quả, async giúp thực hiện điều này:

```
fun fetchData() = viewModelScope.launch {
    val data1 = async { fetchFirstData() }
    val data2 = async { fetchSecondData() }
    // Chờ cả hai tác vụ hoàn thành
    val combinedData = data1.await() + data2.await()
    updateUI(combinedData)
}
```

Xử lý ngoại lệ với Coroutine

Coroutine cung cấp cách xử lý ngoại lệ trực tiếp thông qua try-catch:

```
fun fetchData() = viewModelScope.launch {
    try {
        val data = fetchFromNetwork()
        updateUI(data)
    } catch (e: Exception) {
        // Xử lý ngoại lệ
        showError(e.message)
    }
}
```

Sử dụng coroutines giúp việc lập trình không đồng bộ trở nên đơn giản và dễ dàng hơn. Khi sử dụng đúng cách với các CoroutineScope như viewModelScope hoặc lifecycleScope, có thể quản lý tốt luồng dữ liệu trong ứng dụng mà không lo về rò rỉ bộ nhớ hoặc "đóng băng" giao diện người dùng.

Json

JSON (JavaScript Object Notation) là định dạng dữ liệu độc lập và là sự chuyển đổi tốt nhất của định dạng XML. Phần này sẽ trình bày cú pháp của file JSON và cách trích xuất các thông tin cần thiết từ nó.

Android cung cấp các lớp khác nhau để làm việc trên dữ liệu JSON là SONArray, JSONObject, JSONStringer và JSONTokenizer.

Một file JSON có định dạng gồm các trường thông tin, mỗi trường gồm các thuộc tính khác nhau, chẳng hạn file JSON gồm 3 trường như sau:

```
{
"sys":
{
"country":"GB", "sunrise":1381107633, "sunset":1381149604
},
"weather":[
{
"id":711,
```

```
"main":"Smoke", "description":"smoke", "icon":"50n"
}
],
"main":
{
 "temp":304.15,
"pressure":1009,
}
}
```


Một file JSON gồm nhiều thành phần, bảng sau đây định nghĩa các thành phần được ký hiệu như sau:

Các thành phần	Mô tả
[	Biểu diễn một mảng
{	Biểu diễn một đối tượng
key/value	Đại diện cho một cặp thuộc tính và giá trị của đối tượng; các giá trị có thể nhận kiểu khác nhau như String, int, float,. . .

Bảng 5.1: Các ký hiệu trong file Json

Khởi tạo đối tượng file JSON và chỉ định đối tượng nhận dữ liệu từ nó:

String in;

JSONObject reader = new JSONObject(in); Đọc dữ liệu từ các đối tượng trong file JSON:

// Đọc đối tượng có tên là sys

JSONObject sys = reader.getJSONObject("sys");

// Lấy giá trị của thuộc tính country country = sys.getString("country");

// Đọc đối tượng có tên là main

JSONObject main = reader.getJSONObject("main");

// Lấy giá trị của thuộc tính temp temperature = main.getString("temp");

Một số phương thức của lớp JSONObject và ý nghĩa của chúng được liệt kê trong bảng 4.2

Thí dụ 3.6. Thiết kế ứng dụng Android cho phép truy thông tin về các contact chứa trong một file JSON từ Server trên mạng tại địa chỉ http://api.androidhive.info/contacts/ như sau:

Phương thfíc	Mô tả
get(String name)	Trả về giá trị của thuộc tính được chỉ định
getBoolean(String name)	Trả về giá trị kiểu Boolean của thuộc tính được chỉ định
getDouble(String name)	Trả về giá trị kiểu Double của thuộc tính được chỉ định
getInt(String name)	Trả về giá trị kiểu Int của thuộc tính được chỉ định

getLong(String name)	Trả về giá trị kiểu Long của thuộc tính được chỉ định
length()	Trả về số lượng cặp thuộc tính/giá trị của đối tượng
names()	Trả về một mảng chứa tên các thuộc tính của đối tượng

Bảng 5.2: Các phương thức của lớp JSONObject

Hình 8.7: Ứng dụng truy xuất file JSON

Cấu trúc file JSON chứa một mảng các contact, mỗi contact gồm các thông tin có dạng như sau:

```
{
"contacts": [
{
"id": "c200",
"name": "Ravi Tamada", "email": "ravi@gmail.com",
 "address": "xx-xx-xxxx,x - street, x - country", "gender" : "male",
"phone": {
"mobile": "+91 0000000000",
"home": "00 000000",
"office": "00 000000"
}
},
{
"id": "c201",
"name": "Johnny Depp",
"email": "johnny_depp@gmail.com",
"address": "xx-xx-xxxx,x - street, x - country", "gender" : "male",
"phone": {
"mobile": "+91 0000000000",
"home": "00 000000",
"office": "00 000000"
}
},
....
]
```

```
    }
```

Lấy dữ liệu trên Internet

Phần cấu hình

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <ImageView
        android:id="@+id/iv"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:src="@drawable/ic_launcher_background"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

Phần xử lý với thư viện Retrofit

```
package com.example.cnpm_k20_n01_tuan7
import com.google.gson.annotations.SerializedName
data class Image(
    @SerializedName("id") val id: String,
    @SerializedName("img_src") val imgSrc: String
)

package com.example.cnpm_k20_n01_tuan7
import retrofit2.http.GET
interface MyService {
    @GET("photos")
    suspend fun getImage(): List<Image>

}

package com.example.cnpm_k20_n01_tuan7
import retrofit2.Retrofit
import retrofit2.converter.gson.GsonConverterFactory
object MyBuilder{
    fun getRetro(): Retrofit {
        return Retrofit.Builder()
```

```
            .baseUrl("https://android-kotlin-fun-mars-server.appspot.com/")
            .addConverterFactory(GsonConverterFactory.create())
            .build()
    }
}

package com.example.cnpm_k20_n01_tuan7
import android.os.Bundle
import android.util.Log
import android.widget.ImageView
import androidx.activity.enableEdgeToEdge
import androidx.appcompat.app.AppCompatActivity
import androidx.core.view.ViewCompat
import androidx.core.view.WindowInsetsCompat
import com.bumptech.glide.Glide
import kotlinx.coroutines.CoroutineScope
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.GlobalScope
import kotlinx.coroutines.delay
import kotlinx.coroutines.launch
import kotlinx.coroutines.withContext
import java.io.InputStreamReader
import java.net.HttpURLConnection
import java.net.URL
class MainActivity : AppCompatActivity() {
    lateinit var iv : ImageView
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContentView(R.layout.activity_main)
        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main))
{ v, insets ->
            val systemBars =
insets.getInsets(WindowInsetsCompat.Type.systemBars())
            v.setPadding(systemBars.left, systemBars.top, systemBars.right,
systemBars.bottom)
            insets
        }
        iv = findViewById(R.id.iv)
        //Tao ket noi
//        var list = listOf<String>()
        var list = listOf<Image>()
        CoroutineScope(Dispatchers.Default).launch() {
            withContext(Dispatchers.IO){
//                val url = URL("https://android-kotlin-fun-mars-
server.appspot.com/photos")
//                val con = url.openConnection() as HttpURLConnection
```

```
//                  con.doInput
//                  con.doOutput
//                  //tao luong
//                  val input = con.inputStream
//                  val reader = InputStreamReader(input)
//                  //xly dl
//                  list = reader.readLines()
//                  //dong ket noi
//                  reader.close()
//                  input.close()
//                  con.disconnect()
                  //dung retrofit
                  val con =
                  MyBuilder.getRetro().create(MyService::class.java)
                  list = con.getImage()
              }

              withContext(Dispatchers.Main){
//                  list.forEach {
//                      Log.d("AAA", it)
//                      var ivUrl=""
//                      if(it.contains("https:")){
//                          ivUrl = it.substring(it.indexOf("https:"),
                                  it.length-1)
//                          //load anh tu ivUrl
//                          Glide.with(applicationContext)
//                              .load(ivUrl)
                            .placeholder(R.drawable.ic_launcher_background)

//                          .error(R.drawable.ic_launcher_foreground) // Hiển
                             thị ảnh lỗi nếu có lỗi
//                              .into(iv)
//                          delay(5000)
//                      }
                  list.forEach{
                               Glide.with(applicationContext)
                              .load(it.imgSrc)
                              .placeholder(R.drawable.ic_launcher_background)
                              .error(R.drawable.ic_launcher_foreground)
                              .into(iv)
                          delay(5000)
                  }
              }
          }

      }
}
```

8.4. UDF và Firebase trong Android

Trong Unidirectional Data Flow (UDF), dữ liệu di chuyển theo một hướng từ nguồn đến giao diện người dùng, trong khi các sự kiện do người dùng kích hoạt sẽ truyền ngược lại để cập nhật dữ liệu. Khi sử dụng Firebase trong ứng dụng Android, mô hình này vẫn được áp dụng để đảm bảo tính nhất quán và quản lý trạng thái hiệu quả.

1. Firebase là gì?

Firebase là một nền tảng Backend-as-a-Service (BaaS) cung cấp các công cụ mạnh mẽ để lưu trữ, đồng bộ và quản lý dữ liệu trên nền tảng đám mây. Một số dịch vụ phổ biến của Firebase bao gồm:

Firebase Realtime Database và Cloud Firestore: Cơ sở dữ liệu NoSQL hỗ trợ đồng bộ hóa dữ liệu theo thời gian thực.

Firebase Authentication: Xác thực người dùng với Google, Facebook, Email, v.v.

Firebase Cloud Messaging (FCM): Gửi thông báo đẩy đến ứng dụng Android.

Firebase Storage: Lưu trữ tệp như hình ảnh, video.

Firebase Analytics: Thu thập và phân tích hành vi người dùng.

2. Kết hợp Firebase với UDF

Khi sử dụng Firebase trong ứng dụng Android theo mô hình UDF, luồng dữ liệu có thể được tổ chức như sau:

Dữ liệu từ Firebase chảy xuống UI:

Repository lấy dữ liệu từ Cloud Firestore hoặc Realtime Database.

ViewModel xử lý dữ liệu và phát nó dưới dạng LiveData hoặc StateFlow.

UI (Activity/Fragment/Compose) lắng nghe sự thay đổi từ ViewModel và hiển thị dữ liệu mới nhất.

Sự kiện người dùng chảy ngược lên để cập nhật Firebase:

Người dùng nhấn nút "Lưu" → UI gửi sự kiện đến ViewModel.

ViewModel gọi Repository để cập nhật dữ liệu trong Firebase.

Firebase cập nhật dữ liệu và tự động đồng bộ với ViewModel để phản ánh trạng thái mới lên UI.

3. Ví dụ: Lưu và hiển thị danh sách công việc từ Firebase

1. Repository – Tương tác với Firestore

```
class TaskRepository {
    private val db = FirebaseFirestore.getInstance()
    private val tasksCollection = db.collection("tasks")

    fun getTasks(): Flow<List<Task>> = callbackFlow {
        val listener = tasksCollection.addSnapshotListener { snapshot,
       _ ->
            snapshot?.let {
```

```
                val tasks = it.toObjects(Task::class.java)
                trySend(tasks)
            }
        }
        awaitClose { listener.remove() }
    }

    suspend fun addTask(task: Task) {
        tasksCollection.add(task)
    }
}
```

2. ViewModel – Quản lý dữ liệu

```
class TaskViewModel(private val repository: TaskRepository) :
 ViewModel() {
    val tasks: StateFlow<List<Task>> = repository.getTasks()
        .stateIn(viewModelScope, SharingStarted.Lazily, emptyList())

    fun addTask(task: Task) {
        viewModelScope.launch {
            repository.addTask(task)
        }
    }
}
```

3. UI – Hiển thị dữ liệu

```
class TaskActivity : AppCompatActivity() {
    private val viewModel: TaskViewModel by viewModels()

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            val tasks by viewModel.tasks.collectAsState()
            Column {
                LazyColumn {
                    items(tasks) { task ->
                        Text(task.name)
                    }
                }
                Button(onClick = { viewModel.addTask(Task("New Task"))
                   }) {
                    Text("Add Task")
                }
            }
        }
    }
}
```

4. Kết luận

Firebase kết hợp với UDF giúp ứng dụng Android có kiến trúc rõ ràng, dữ liệu đồng bộ theo thời gian thực mà không cần cập nhật thủ công. Việc sử dụng LiveData/StateFlow và ViewModel giúp đảm bảo dữ liệu luôn nhất quán và dễ dàng mở rộng.

Bài tập bổ trợ:

Bài tập thực hành cơ bản

1. Gọi API lấy dữ liệu JSON và hiển thị trong RecyclerView.
2. Sử dụng Retrofit để gửi yêu cầu POST với thông tin người dùng.
3. Sử dụng Glide để tải và hiển thị hình ảnh từ URL.
4. Quản lý trạng thái tải dữ liệu bằng Coroutine hoặc LiveData.
5. Thêm tính năng lưu trữ ngoại tuyến cho ứng dụng gọi API.

Bài mẫu: Gọi API và hiển thị dữ liệu

Mục tiêu: Tạo ứng dụng gọi API và hiển thị danh sách bài viết từ JSON trong RecyclerView.

Bước thực hiện:

Thêm Dependency Retrofit:

```
implementation "com.squareup.retrofit2:retrofit:2.9.0"
implementation "com.squareup.retrofit2:converter-gson:2.9.0"
```

Tạo API Service:

```
import retrofit2.Retrofit
import retrofit2.converter.gson.GsonConverterFactory
import retrofit2.http.GET

data class Post(val id: Int, val title: String)

interface ApiService {
    @GET("posts")
    suspend fun getPosts(): List<Post>
}

val retrofit = Retrofit.Builder()
    .baseUrl("https://jsonplaceholder.typicode.com/")
    .addConverterFactory(GsonConverterFactory.create())
    .build()

val apiService = retrofit.create(ApiService::class.java)
```

Hiển thị dữ liệu trong RecyclerView:

Adapter:

```
class PostAdapter(private val posts: List<Post>) :
```

```
    RecyclerView.Adapter<PostAdapter.PostViewHolder>() {

    class PostViewHolder(val view: View) : RecyclerView.ViewHolder(view)

    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int):
PostViewHolder {
        val view = LayoutInflater.from(parent.context).inflate(R.layout.item_post,
parent, false)
        return PostViewHolder(view)
    }

    override fun onBindViewHolder(holder: PostViewHolder, position: Int) {
        holder.view.textViewTitle.text = posts[position].title
    }

    override fun getItemCount() = posts.size
}
```

Activity/Fragment:

```
lifecycleScope.launch {
    val posts = apiService.getPosts()
    recyclerView.adapter = PostAdapter(posts)
}
```